

Systèmes d'exploitation I

Module: M3 - INF-S3 & IAP-S3

Prof. R. HANNANE

r.hannane@uca.ma

Faculté des Sciences Senglalia, Marrakech
Université Cadi Ayyad



Connexion/Déconnexion d'un utilisateur en Linux

Connexion d'un utilisateur

- Le système Linux étant un **système multi-utilisateur et multitâche** : $\Rightarrow$ plusieurs personnes sont connectées simultanément et peuvent travailler sans interférer les unes avec les autres
- Linux met en œuvre des mécanismes **d'identification des utilisateurs**, tout en permettant le **partage contrôlé** nécessaire au travail en groupe
- Tout utilisateur est **identifié par un nom** (*login name*)
- Cet utilisateur ne peut utiliser le système que si son nom a préalablement été défini par **l'administrateur du système** (ou super-utilisateur), dont le nom est généralement **root**

Connexion d'un utilisateur

- Lorsque on installe Linux, et pour ouvrir une session, on a l'opportunité d'indiquer si on veut utiliser un **écran graphique** (**connexion en mode graphique**) plutôt qu'une **console** (**connexion en mode texte**)
- Pour qu'un utilisateur puisse travailler avec le système Linux, **il doit établir une connexion qui consiste à s'identifier auprès du système** ⇒ **ouvrir une session**:
 - ▶ **Login**: saisir le nom d'utilisateur
 - ▶ **Password**: saisir son mot de passe

Attention:

- **Les caractères saisis ne sont pas affichés à l'écran** (on dit qu'il n'y a **pas d'écho des caractères sur le terminal**)
- Ce mécanisme permet de garder la confidentialité du mot de passe

Connexion d'un utilisateur

- En cas d'erreur lors de la saisie du nom ou du mot de passe, le système donne à l'utilisateur la possibilité de recommencer
- Si la connexion est réussie, il apparaît alors à l'écran un **certain nombre d'informations** (informations générales, date, arrivée de messages, date de dernière connexion)
- Puis le système lance un programme qui est généralement un **interpréteur de commandes**

Déconnexion d'un utilisateur

- Pour sortir de la session, on peut :
 - Utiliser la commande `exit` si on est sur un environnement de lignes de commande
⇒ *La déconnexion est effective lorsqu'apparaît l'invite login :*
 - Sélectionner avec le bouton gauche de la souris le menu démarrer puis choisir la commande `log out`
⇒ *La déconnexion est effective lorsqu'apparaîtra l'invite graphique login*

Connexion/Déconnexion d'un utilisateur

Conseils généraux

Remarques:

- Si vous avez réalisé vous-même l'installation, une fois cette dernière terminée, **connectez-vous au système en tant qu'administrateur** (répondre root au login :)
- Il faut immédiatement définir un mot de passe pour cet utilisateur root (Ce mot de passe doit contenir plus de six caractères)
- Ce mot de passe vous permettra de vous connecter en tant qu'utilisateur **root** et ainsi d'avoir tous les droits sur le système

Connexion/Déconnexion d'un utilisateur

Conseils généraux

Attention:

- Le compte **root** est réservé pour le superviseur du système
- **Il ne faut jamais utiliser root comme un compte personnel**
- Il doit être utilisé pour réaliser des modifications dans **votre système** $\Rightarrow$ *C'est pourquoi la première action d'un administrateur est de créer un compte personnel*
- Un utilisateur peut à tout moment changer son mot de passe mais, il faut fournir l'ancien

L'interpréteur de commandes

- Après l'ouverture de la session, l'utilisateur se trouve sous le contrôle d'un **interpréteur de commandes**
- C'est un logiciel système faisant partie des composants de base d'un SE
- Sa fonction est d'interpréter les commandes qu'un utilisateur tape au clavier dans l'interface en ligne de commande
⇒ **Shell** est l'interpréteur qui traduit les commandes entrées par l'utilisateur en actions que le noyau doit exécuter
- Il existe plusieurs types de Shell mais le plus utilisé par les distributions Linux est **bash**
- Cet interpréteur de commandes est prêt à lire, analyser et éventuellement exécuter les commandes qui lui sont soumises

L'interpréteur de commandes

- **L'interpréteur indique par une chaîne de caractères** (marqueur au début de ligne), appelée **invite** (ou **prompt**), qu'il est prêt à recevoir une commande
- **Le marqueur du début de ligne est variable selon les machines**

Exemple: \$ ou *nom_utilisateur@nom_machine* >

```
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~$ █
```

- **Le caractère de terminaison peut avoir d'autres significations:**
 - un \$ indique que **l'utilisateur n'a pas de pouvoirs particuliers**, comme le >
 - un # indique que **l'utilisateur est l'administrateur root qui a tous les pouvoirs**

L'interpréteur de commandes

Répertoire de connexion

- Après le login, **l'interpréteur de commande** a pour répertoire courant le **répertoire de connexion** de l'utilisateur
- Sous le répertoire racine **“/”** on trouve un répertoire appelé **“home”**
- Sous le répertoire **“home”** il y aura un **répertoire pour chaque utilisateur dans le système**
- Ce répertoire qui porte le nom de l'utilisateur est appelé **répertoire personnel de l'utilisateur** ou (home directory)
Exemple : Un utilisateur appelé “abdessamad” aura comme répertoire personnel **“/home/abdessamad”**

L'interpréteur de commandes

Répertoire de connexion

- **L'utilisateur a un contrôle total sur son répertoire personnel**
⇒ L'utilisateur peut y placer ses fichiers personnels, et créer autant de sous-répertoires qu'il le désire
- **Vous pouvez représenter votre répertoire personnel par le symbole "*~*" dans l'invite de commande au lieu d'utiliser le chemin complet**
Si votre répertoire personnel est */home/admin* vous pouvez l'écrire aussi *~*
- Pour représenter le répertoire personnel d'un autre utilisateur on peut taper *~user* (*user est le nom de l'utilisateur*)
Par exemple : *~abdessamad* représente le répertoire personnel de abdessamad (*/home/abdessamad*)

Les commandes Linux

Les commandes Linux

Format de commande

- Il existe un très grand nombre de commandes sous Linux et qui sont exécutées depuis l'invite du **shell**
- Syntaxe d'une commande est:
`nom_commande [options] [arguments]`
- Les "**options**" sert à modifier le comportement de la commande, elle **commencent par le caractère - (signe moins) suivi de plusieurs lettre clés** (le plus souvent une seule lettre mais parfois elle peut-être un mot)
 - ▶ Les **options** avec une seule lettre sont précédées par un tiret -
 - ▶ Les **options** avec un mot entier sont précédée par double tiret - -
- Les "**arguments**" spécifient les objets (fichiers ou variables) sur les quels la commande va s'appliquer

Les commandes Linux

Format de commande

- Chaque argument et chaque option doit être séparées des autres par des espaces
- Certaines commandes n'exigent ni arguments ni options
⇒ Dans ce cas, elle exécute l'action par défaut pour laquelle elle est programmée, ou affiche un message d'erreur si ceux-ci sont nécessaires
- Note: Linux fait la différence entre le majuscule et le minuscule pour les commandes , options et arguments

Les commandes Linux

Répertoire courant

- Votre **répertoire courant** est le répertoire dans lequel vous travaillez actuellement dans un terminal
- Cet emplacement est affiché par le prompt
- Par défaut le **répertoire courant** est votre **répertoire personnel** mais on peut le changer
- Pour afficher votre emplacement (répertoire) courant on utilise la commande : **pwd**
- **pwd** : (Print Working Directory) **Affiche le chemin d'accès du répertoire courant**. Juste après connexion d'un utilisateur, cette commande lui précisera son répertoire d'accueil

```
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~$ pwd  
/home/abdessamad
```

Les commandes Linux

- Lorsque l'utilisateur saisit une commande dans le terminal texte, l'interpréteur de commandes suit un ensemble des étapes pour l'exécuter
- **Exemple:** si la commande *allo* à été saisie par l'utilisateur

```
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~$ allo
```

```
Command 'allo' not found, did you mean:
```

```
  command 'aldo' from snap aldo (0.7.7)
```

```
  command 'aldo' from deb aldo (0.7.7-1build2)
```

```
See 'snap info <snapname>' for additional versions.
```

```
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~$
```

Les commandes Linux

La procédure suivie par l'interpréteur de commandes

- ▶ Si `allo` est l'une de ses commandes internes, il l'exécute puis redonne la main à l'utilisateur
- ▶ Sinon, il passe à l'étape suivante:
 - Il parcourt le contenu d'une variable d'environnement appelée **PATH**
⇒ Le contenu de **PATH** est une **liste de répertoires** dans lesquels il faut chercher les binaires correspondant à la commande à exécuter
Exemple, si **PATH** contient les répertoires suivants (dans l'ordre) :
`/usr/bin; /bin; /home/user/bin` alors l'interpréteur de commandes va chercher les commandes : `/usr/bin/allo; /bin/allo; /home/user/bin/allo`
 - Si aucun de ces répertoires ne contient la commandes `allo`, l'interpréteur de commandes affiche un message d'erreur pour indiquer à l'utilisateur qu'il ne connaît pas cette commande

Les commandes Linux

Types de Commandes

- Il existe **deux types de commandes** :
 - ▶ Les **commandes internes** : sont internes au shell et exécutées au sein de celui-ci. Ces commandes font partie du programme shell, le **bash**. Quand on les exécute, le shell exécute les fonctions définies en son sein correspondant à celles-ci
Exemple: La commande **pwd**
 - ▶ Les **commandes externes** : sont des programmes binaires présents en tant que fichiers sur le disque dur (ou tout autre support de données). Quand on exécute la commande, ce fichier est chargé en mémoire et lancé en tant que processus
Exemple: La commande **date**

Les commandes Linux

Types de Commandes

- On peut **distinguer une commande interne d'une commande externe** à l'aide de la commande interne **type**

⇒ C'est ainsi que **date** est une commande externe, on constate que c'est un fichier présent dans */bin*, tandis que **pwd** est interne au shell

```
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~$ type cd
cd is a shell builtin
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~$ type date
date is hashed (/usr/bin/date)
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~$ type pwd
pwd is a shell builtin
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~$
```

Les commandes Linux

Types de Commandes

- On peut tomber sur certains **autres types de commandes** comme les **alias** qui sont des raccourcis de commandes propres au shell
- Ainsi le shell bash de certaines distributions Linux proposent des alias comme la commande **ll** qui correspond en fait à la commande **ls -aF**

```
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~$ type ll
ll is aliased to `ls -aF`
```

Les commandes Linux

- Le **Shell bash** offre plusieurs fonctionnalités :
 - 1 **Alias**: Permet de **donner des noms réduits** à des commandes dont le nom est long
 - 2 **Variables**: Permettent de **stocker des informations** utilisées par le Shell bash
 - 3 **Historique des commandes**: Permet d'**accéder facilement** aux commandes déjà exécutées et les réexécuter
 - 4 **Scripting**: Un script shell est un **fichier exécutable** contenant **plusieurs commandes**, son rôle est d'**automatiser l'exécution** de plusieurs commandes en une seule (le script)

Les commandes Linux

1) Alias du shell bash

- Un **alias** est utilisé pour remplacer une longue commande par un pseudo plus petit
- De cette façon l'utilisateur n'aura pas à taper de longue commandes

Exemple: la commande `ls -l` a souvent comme alias `l` ou `ll`

- Pour visualiser les alias définis sur le système on utilise la commande `alias`
- Pour créer de nouveaux alias on utilise la commande :
`alias nom=commande`

Attention:

- Il faut noter que les alias créés de cette façon ne sont pas permanents et seront perdus une fois le Shell fermé

Les commandes Linux

2) Variables du shell bash

- Une **variable** permet au Shell de stocker des données qui peuvent être utilisée par le Shell ou par une commande
 - Une variable **possède un nom et est stockée temporairement dans la mémoire une fois le terminal ferme toute les variables créés seront perdues**
- ⇒ **Exemple:** la variable **HISTSIZE** contient le nombre de commandes mémorisées dans l'historique

Les commandes Linux

2) Variables du shell bash

- Pour afficher à l'écran du texte ou la valeur d'une variable on utilise la commande **echo**

⇒ Exemple d'affichage du texte:

```
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~$ echo Bonjour tout le monde
Bonjour tout le monde
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~$
```

⇒ Exemple d'affichage de la valeur d'une variable:

```
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~/Desktop/SE1/test$ echo $HISTSIZE
1000
```

Attention: Il faut faire précéder le nom de la variable par un **\$**

- Pour modifier la valeur d'une variable il ne faut pas utiliser le signe **\$**

⇒ Exemple: **HISTSIZE=500**

Les commandes Linux

2) Variables du shell bash

- La variable **PATH** indique au Shell les emplacement ou chercher les commandes à exécuter
- Si on tape une commande et on reçoit l'erreur "command not found" c'est parce que le **Shell** n'arrive pas à trouver la commande saisie dans les chemins indiqués par la variable **PATH**

```
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~/Desktop/SE1/test$ echo $PATH
/home/abdessamad/.local/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~/Desktop/SE1/test$
```

Les commandes Linux

3) L'historique des commandes

- Lorsqu'on exécute une commande sur un terminal celle-ci est enregistrée dans une "liste d'historique des commandes"
- Plus on tape des commandes, plus l'historique s'agrandit
⇒ Le shell conserve ainsi un grand nombre d'entrées dans l'historique
- Cela s'avère intéressant lorsqu'on veut exécuter la même commande une autre fois ⇒ on n'aura pas à la resaisir
- Cet historique est conservé dans un fichier caché du répertoire personnel appelé *.bash_history*

Les commandes Linux

3) L'historique des commandes

- C'est très **utile de pouvoir rappeler une commande qu'on a déjà exécutée en naviguant dans l'historique des commandes**
⇒ C'est avec les touches $\uparrow$ [*Flèche en haut*] et $\downarrow$ [*Flèche en bas*]
- La **flèche du haut** remonte dans l'historique:
Si on a saisi les deux précédentes commandes (**echo** puis **pwd**)
le premier appui sur la flèche du haut affiche la ligne de commande **pwd**
un second appui affichera la commande **echo**
- La **flèche du bas** navigue dans l'autre sens, jusqu'à l'invite d'origine
- Si on appuie sur la touche [*Entrée*], on lance de nouveau la commande

Les commandes Linux

3) L'historique des commandes

- Pour afficher la liste d'historique on utilise la commande `history`

```
260 pip3 --version
261 pip3 list
262 clear
263 python3 -m pip show scikit-learn
264 exit
265 ls
266 cd
267 allo
268 ls /etc
269 ls /etc/ppp
270 ls /etc/ssh
271 ls /etc/ppp /etc/ssh
272 ls -l
273 ls -h
274 ls -l -h
275 ls --human-readable
276 pwd
277 date
278 history
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~$
```

Les commandes Linux

3) L'historique des commandes

- La commande `fc` effectue presque la même chose lorsqu'on utilise le paramètre `-l`

```
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~$ fc -l
263      python3 -m pip show scikit-learn
264      exit
265      ls
266      cd
267      allo
268      ls /etc
269      ls /etc/ppp
270      ls /etc/ssh
271      ls /etc/ppp /etc/ssh
272      ls -l
273      ls -h
274      ls -l -h
275      ls --human-readable
276      pwd
277      date
278      history
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~$
```

Les commandes Linux

3) L'historique des commandes

- Par défaut la commande `fc` se limite aux 16 dernières commandes, mais $\Rightarrow$ On peut lui passer le nombre des dernières commandes à afficher
- Exemple: l'historique des 5 dernières commandes

```
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~$ fc -l -5
```

```
275      ls --human-readable
```

```
276      pwd
```

```
277      date
```

```
278      history
```

```
279      fc -l
```

```
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~$
```

```
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~$ history 5
```

```
279  fc -l
```

```
280  fc -l -5
```

```
281  sudo pam-auth-update
```

```
282  date
```

```
283  history 5
```

```
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~$
```

Les commandes Linux

3) L'historique des commandes

- Pour exécuter une commande qui figure dans l'historique on tape dans la ligne de commande un point d'exclamation suivi par son numéro d'ordre :

Exemple: **!11**

- Autre options:

Exemple	signification
history 5	Afficher les 5 dernières commandes dans l'historique
!!	Exécuter la dernière commande une autre fois
!-5	Exécuter la 5 ^{ème} commande partant de la fin de la liste
!ls	Exécuter la dernière commande ls

Les commandes Linux

3) L'historique des commandes

```
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~$ !277
date
Fri 22 Oct 2021 03:31:29 PM +01
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~$
```

```
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~$ !-5
fc -l
268      ls /etc
269      ls /etc/ppp
270      ls /etc/ssh
271      ls /etc/ppp /etc/ssh
272      ls -l
273      ls -h
274      ls -l -h
275      ls --human-readable
276      pwd
277      date
278      history
279      fc -l
280      fc -l -5
281      sudo pam-auth-update
282      date
283      history 5
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~$
```

Les commandes Linux

Utilisation des sources d'aide

- Le Shell Linux offre plusieurs **sources d'aides** permettant de **comprendre le comportement des commandes** et aussi de **découvrir de nouvelles commandes**
- Parmi les sources d'aides :
 - 1) Les pages **man**
 - 2) L'option **--help**
 - 3) Les commandes d'aide (**whatis** , **whereis** et **which**)

Les commandes Linux

Utilisation des sources d'aide: 1) Les pages `man`

- Le manuel (`man`) permet de **donner une description détaillée d'une commande avec toutes ses options**
- Pour afficher les pages `man` d'une commande il faut taper :
`man commande`

⇒ Exemple : `man cal` permet d'afficher les pages du manuel concernant la commande `cal`

```
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~$ man cal
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~$ man mkdir
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~$
```

Les commandes Linux

Utilisation des sources d'aide: 1) Les pages `man`: La navigation

- Le tableau suivant indique les raccourcis clavier permettant de naviguer à travers une page du manuel (`man`)

Commande / touche clavier	rôle
entrée	Afficher la ligne suivante
espace	Afficher la page suivante
/terme	Chercher un terme
n	Trouver le prochain terme recherché
1G	Aller au début du manuel
G	Aller à la fin du manuel
h	Afficher l'aide
q	Quitter le manuel

Les commandes Linux

Utilisation des sources d'aide: 1) **Les pages man**: Ces sections

- Les pages **man** sont **subdiviser en section** chaque section donne une description de la commande
- Le tableau suivant résume les principales sections qu'on peut trouver dans une page **man**

Section	Informations fournies
NAME	Donne le nom de la commande et une très brève description
SYNOPSIS	Donne la syntaxe de la commande
DESCRIPTION	Donne plus de détaille sur la commande
OPTIONS	Donne la description de chacune des options de la commande, parfois ces informations figurent dans la section Description et non dans la section Options
FILES	Donne l'ensemble des fichiers associés à la commande et qui permettent de configurer la commande
AUTHOR	Le nom et parfois le contact de la personne qui a crée la page man
REPORTING BUGS	Comment reporter les bugs relatifs à la commande
COPYRIGHT	Information sur le copyright (droit d'auteur)
SEE ALSO	Où chercher d'autres informations ou quelles sont les commandes associées à la commande en question

Les commandes Linux

Utilisation des sources d'aide: 1) Les pages `man`: Ces sections

```

CAL(1)                                BSD General Commands Manual                                CAL(1)

NAME
    cal, ncal – displays a calendar and the date of Easter

SYNOPSIS
    cal [-31jy] [-A number] [-B number] [-d yyyy-mm] [[month] year]
    cal [-31j] [-A number] [-B number] [-d yyyy-mm] -m month [year]
    ncal [-C] [-31jy] [-A number] [-B number] [-d yyyy-mm] [[month] year]
    ncal [-C] [-31j] [-A number] [-B number] [-d yyyy-mm] -m month [year]
    ncal [-31bhjpwysM] [-A number] [-B number] [-H yyyy-mm-dd] [-d yyyy-mm] [-s country_code] [[month]
        year]
    ncal [-31bhJeoSM] [-A number] [-B number] [-d yyyy-mm] [year]

DESCRIPTION
    The cal utility displays a simple calendar in traditional format and ncal offers an alternative lay-
    out, more options and the date of Easter. The new format is a little cramped but it makes a year
    fit on a 25x80 terminal. If arguments are not specified, the current month is displayed.

    The options are as follows:

    -h      Turns off highlighting of today.

    -J      Display Julian Calendar, if combined with the -o option, display date of Orthodox Easter ac-
    cording to the Julian Calendar.

    -e      Display date of Easter (for western churches).

    -j      Display Julian days (days one-based, numbered from January 1).

    -m month
        Display the specified month. If month is specified as a decimal number, appending 'f' or

```

Les commandes Linux

Utilisation des sources d'aide: 2) **L'option --help**

- Plusieurs commandes possèdent une option **--help** permettant d'**afficher une aide sommaire sur la commande et ses options**

```

abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~$ cd --help
cd: cd [-L|[-P [-e]] [-@]] [dir]
    Change the shell working directory.

    Change the current directory to DIR.  The default DIR is the value of the
    HOME shell variable.

    The variable CDPATH defines the search path for the directory containing
    DIR.  Alternative directory names in CDPATH are separated by a colon (:).
    A null directory name is the same as the current directory.  If DIR begins
    with a slash (/), then CDPATH is not used.

    If the directory is not found, and the shell option 'cdable_vars' is set,
    the word is assumed to be a variable name.  If that variable has a value,
    its value is used for DIR.

Options:
  -L      force symbolic links to be followed: resolve symbolic
          links in DIR after processing instances of '..'
  -P      use the physical directory structure without following
          symbolic links: resolve symbolic links in DIR before
          processing instances of '..'
  -e      if the -P option is supplied, and the current working
          directory cannot be determined successfully, exit with
          a non-zero status
  -@      on systems that support it, present a file with extended

```

Les commandes Linux

Utilisation des sources d'aide: **3) Les commandes d'aide:** `whatis`

- `whatis` est une commande qui permet de **chercher et d'afficher les pages du manuel relatifs à un mot clé avec une description courte de chaque page**

```
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~$ whatis ls
ls (1)                - list directory contents
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~$ whatis ps
ps (1)                - report a snapshot of the current processes.
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~$ whatis cd
cd: nothing appropriate.
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~$
```

Les commandes Linux

Utilisation des sources d'aide: **3) Les commandes d'aide:** `whereis`

- `whereis` est une commande qui permet d'**afficher l'emplacement des commandes et des pages `man` relatives à une commande**

```
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~$ whereis ls
ls: /usr/bin/ls /usr/share/man/man1/ls.1.gz
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~$ whereis ps
ps: /usr/bin/ps /usr/share/man/man1/ps.1.gz
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~$ whereis cd
cd:
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~$
```

Les commandes Linux

Utilisation des sources d'aide: **3) Les commandes d'aide:** `which`

- `which` est une commande qui permet **d'afficher le chemin où est stocké une commande donnée** ⇒ Elle cherche dans la variable `PATH` pour déterminer le chemin d'une commande

```
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~/Desktop/SE1/test$ which echo
/usr/bin/echo
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~/Desktop/SE1/test$ which mkdir
/usr/bin/mkdir
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~/Desktop/SE1/test$ which rmdir
/usr/bin/rmdir
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~/Desktop/SE1/test$
```